

Three essential modern JavaScript features: safe default values, safe property access, and the most important array methods for data transformation.

Part 1 — Nullish Coalescing Operator ??

The Problem — Falsy Values

The logical OR `||` operator is commonly used to provide default values:

```
let x = undefined;
x = x || 0;
console.log(x); // 0
```

This works, but `||` has a critical flaw — it triggers the default for **any falsy value**, not just `undefined` or `null`:

```
// All of these trigger the default with ||
let a = 0 || "default"; // "default" - but 0 is a valid
number!
let b = "" || "default"; // "default" - but "" is a valid
string!
let c = false || "default"; // "default" - but false is a valid
boolean!
let d = null || "default"; // "default" - correct
let e = undefined || "default"; // "default" - correct
```

The bug: What if `0`, `""`, or `false` are intentional, valid values? `||` incorrectly replaces them.

The Solution — ?? (Nullish Coalescing)

`??` only triggers the default when the left side is `null` or `undefined` — nothing else:

```

let x = undefined;
x = x ?? 1;
console.log(x); // 1

// The difference:
0      ?? "default"; // 0      - 0 is kept! (not
null/undefined)
""     ?? "default"; // ""     - empty string is kept!
false  ?? "default"; // false  - false is kept!
null   ?? "default"; // "default" - null triggers default
undefined ?? "default"; // "default" - undefined triggers default

```

|| vs ?? — Side-by-Side

```

const score = 0;

const display1 = score || "No score"; // "No score" - WRONG! 0 is
valid
const display2 = score ?? "No score"; // 0 - CORRECT!

```

Value	\ result	?? result
0	default (wrong)	0 (correct)
""	default (wrong)	"" (correct)
false	default (wrong)	false (correct)
null	default	default
undefined	default	default
1	1	1
"hello"	"hello"	"hello"

Rule: Use `??` when your value could legitimately be `0`, `""`, or `false`. Use `||` only when all falsy values should trigger the default.

Nullish Assignment `??=`

A shorthand for assigning only if the variable is null/undefined:

```
let x = null;
x ??= 42;
console.log(x); // 42

let y = 0;
y ??= 42;
console.log(y); // 0 - not replaced, 0 is not null/undefined
```

Part 2 — Optional Chaining `?.`

The Problem — Accessing Nested Properties

When you access a property on an object that might not exist, JavaScript throws a `TypeError`:

```
const user3 = { fname: "mona" }; // no 'address' property

console.log(user3.address.country);
// TypeError: Cannot read properties of undefined (reading 'country')
// Because user3.address is undefined, and undefined.country throws
```

The old defensive solution was verbose:

```
const country = user3 && user3.address && user3.address.country;
// undefined - safe, but ugly
```

The Solution — `?.` (Optional Chaining)

`?.` short-circuits and returns `undefined` if the left side is `null` or `undefined`, instead of throwing an error:

```
const user1 = {
  fname: "mona",
```

```
    address: { city: "cairo", country: "Egypt" },
    greet() { console.log("hello"); }
};

const user2 = { fname: "mona", address: { street: "123" } };
const user3 = { fname: "mona" }; // no address, no greet

console.log(user1.address?.country); // "Egypt" - exists, returned
console.log(user2.address?.country); // undefined - country missing,
no error
console.log(user3.address?.country); // undefined - address missing,
no error
```

No error is thrown. The chain safely returns `undefined` at the first missing link.

Optional Chaining on Methods `?.()`

Call a method that might not exist:

```
console.log(user1.greet?.()); // "hello" - greet exists, called
normally
console.log(user3.greet?.()); // undefined - greet missing, no error
```

Without `?.()`, calling `user3.greet()` would throw: `TypeError: user3.greet is not a function.`

Optional Chaining on Arrays

```
const data = null;
console.log(data?.[0]); // undefined - safe, no error
```

Combining `?.` with `??`

A powerful real-world pattern:

```
const user = { fname: "mona" };

// If address doesn't exist, return "Unknown" instead of undefined
const city = user.address?.city ?? "Unknown";
console.log(city); // "Unknown"
```

? . Short-Circuit Behavior

Once `?.` hits `null` / `undefined`, the rest of the chain is NOT evaluated:

```
let count = 0;
const obj = null;

obj?.method(count++); // count++ is never executed
console.log(count); // 0 - the expression was short-circuited
```

Part 3 — Array API

How Array Methods Work — The Pattern

Most array methods accept a **callback function** that receives three arguments:

```
array.method(function(element, index, originalArray) {
  // return something
});
```

You'll rarely need all three — usually just the first (the element).

`filter()` — Keep Elements That Pass a Test

`filter()` creates a **new array** with only the elements for which the callback returns `true`. The original array is unchanged.

How it Works Internally

```
function myFilter(arr, callback) {  
  let result = [];  
  for (let i = 0; i < arr.length; i++) {  
    if (callback(arr[i], i, arr)) {  
      result.push(arr[i]);  
    }  
  }  
  return result;  
}
```

Examples

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];  
  
// Even numbers  
const evens = numbers.filter(num => num % 2 === 0);  
console.log(evens); // [2, 4, 6, 8]  
  
// Greater than 3  
const big = numbers.filter(num => num > 3);  
console.log(big); // [4, 5, 6, 7, 8]
```

Filtering Objects

```
const users = [  
  { name: "mona", age: 20, active: true },  
  { name: "ahmed", age: 20, active: true },  
  { name: "omar", age: 20, active: false },  
];  
  
// Regular  
const activeUsers = users.filter(user => user.active);  
  
// With destructuring – even cleaner  
const activeUsers = users.filter(({ active }) => active);
```

```
console.log(activeUsers);  
// [{ name: "mona", ... }, { name: "ahmed", ... }]
```

`filter()` always returns a **new array**. If nothing passes, it returns `[]`. It never returns `null`.

map() — Transform Every Element

`map()` creates a **new array** where every element is the result of running the callback on the original element. Same length, different values.

```
const numbers = [1, 2, 3, 4, 5];  
  
// Double each number  
const doubled = numbers.map(num => num * 2);  
console.log(doubled); // [2, 4, 6, 8, 10]  
// Original: [1, 2, 3, 4, 5] – unchanged
```

Transforming Objects

```
const employees = [  
  { name: "mona", salary: 1000 },  
  { name: "ahmed", salary: 2000 },  
  { name: "omar", salary: 3000 },  
];  
  
// Give everyone a 50% raise  
const raised = employees.map(employee => ({  
  ...employee, // copy all existing properties  
  salary: employee.salary * 1.5 // override salary  
}));  
  
console.log(raised);  
// [{ name: "mona", salary: 1500 }, ...]  
// Original employees array – unchanged
```

Important: The code in the lesson mutates the original object directly (`employee.salary *= 1.5`). The safer approach above uses spread to create a new object. Always prefer immutability.

Extracting a Single Property

```
const names = employees.map(emp => emp.name);  
console.log(names); // ["mona", "ahmed", "omar"]
```

filter() vs map() — Key Difference

	filter()	map()
Purpose	Keep or remove elements	Transform elements
Output length	Same or fewer elements	Always same length
Callback returns	true / false (boolean)	The new value
Use when	You want a subset	You want every element changed

every() — Do ALL Elements Pass?

Returns a single `boolean` — `true` only if the callback returns `true` for **every** element:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];  
  
console.log(numbers.every(num => num > 5)); // false - 1,2,3,4,5 are  
not > 5  
console.log(numbers.every(num => num > 0)); // true - all are  
positive
```

Short-circuits: Stops as soon as it finds one element that fails. Efficient.

some() — Do ANY Elements Pass?

Returns `true` if **at least one** element passes the test:

```
console.log(numbers.some(num => num > 5)); // true - 6,7,8 are > 5
console.log(numbers.some(num => num > 10)); // false - none are > 10
```

Short-circuits: Stops as soon as it finds one element that passes. Efficient.

every() VS some()

	every()	some()
Returns <code>true</code> when	ALL pass	AT LEAST ONE passes
Returns <code>false</code> when	ANY fails	ALL fail
Like	AND logic	OR logic

```
// Real-world use
const allAdults = users.every(user => user.age >= 18);
const hasAdmin  = users.some(user => user.role === "admin");
const allPaid   = orders.every(order => order.paid);
const hasErrors = results.some(result => result.error);
```

find() — Get the First Matching Element

Returns the **first element** that passes the test, or `undefined` if none found:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];

console.log(numbers.find(num => num > 5)); // 6 - first element > 5
console.log(numbers.find(num => num > 10)); // undefined - none found
```

Finding Objects

```
const users = [
  { id: 1, name: "mona" },
  { id: 2, name: "ahmed" },
  { id: 3, name: "omar" },
];

const user = users.find(u => u.id === 2);
console.log(user); // { id: 2, name: "ahmed" }
```

findIndex() — Get the Index of the First Match

Same as `find()`, but returns the **index** instead of the element. Returns `-1` if not found:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];

console.log(numbers.findIndex(num => num > 5)); // 5 (index of value 6)
console.log(numbers.findIndex(num => num > 10)); // -1
```

find() vs findIndex()

	find()	findIndex()
Returns	The element itself	The index
Not found	undefined	-1
Use when	You need the value	You need the position

reduce() — Accumulate to a Single Value

`reduce()` processes each element and accumulates a running result:

```
const numbers = [1, 2, 3, 4, 5];
```

```
// Sum all numbers
const sum = numbers.reduce((accumulator, current) => accumulator +
current, 0);
console.log(sum); // 15

// How it works step by step:
// acc=0, cur=1 → 0+1 = 1
// acc=1, cur=2 → 1+2 = 3
// acc=3, cur=3 → 3+3 = 6
// acc=6, cur=4 → 6+4 = 10
// acc=10, cur=5 → 10+5 = 15
```

The second argument `0` is the **initial value** of the accumulator.

```
// Find maximum value
const max = numbers.reduce((acc, cur) => cur > acc ? cur : acc, -
Infinity);

// Count occurrences
const words = ["apple", "banana", "apple", "cherry", "banana",
"apple"];
const count = words.reduce((acc, word) => {
  acc[word] = (acc[word] || 0) + 1;
  return acc;
}, {});
// { apple: 3, banana: 2, cherry: 1 }
```

Complete Array Methods Summary

Method	Returns	Mutates original?	Use case
<code>filter()</code>	New array (subset)	No	Keep matching elements
<code>map()</code>	New array (same length)	No	Transform every element

Method	Returns	Mutates original?	Use case
<code>reduce()</code>	Single value (any type)	No	Accumulate to one result
<code>every()</code>	Boolean	No	Check if all pass
<code>some()</code>	Boolean	No	Check if any pass
<code>find()</code>	Element or undefined	No	Get first matching element
<code>findIndex()</code>	Index or <code>-1</code>	No	Get index of first match

All of these are **non-mutating** — they return new values and leave the original array unchanged.

Chaining Methods

Because `filter()` and `map()` return new arrays, you can chain them:

```
const employees = [
  { name: "mona", active: true, salary: 1000 },
  { name: "ahmed", active: true, salary: 2000 },
  { name: "omar", active: false, salary: 3000 },
];

// Get the raised salaries of active employees only
const result = employees
  .filter(emp => emp.active) // keep active only
  .map(emp => ({ ...emp, salary: emp.salary * 1.5 })); // raise salary

console.log(result);
// [{ name: "mona", active: true, salary: 1500 },
//   { name: "ahmed", active: true, salary: 3000 }]
```

Contact : +201012613453